

DATA STRUCTURES LAB MANUAL

24CSA281

Program 1

Aim: Implement the following array operations on one-dimensional array

- a. Insert
- b. Delete
- c. Display
- d. Search
- e. Add two arrays

Code

```
#include <stdio.h>

#define MAX 100 // maximum size of array

int arr[MAX], n = 0; // array and current size


// Function to insert element at the end

void insert(int val) {

    if (n == MAX) {

        printf("\nArray is full!\n");
```

```
return;

}

arr[n] = val;

n++;

printf("\nInserted %d at the end.\n", val);

}
```

// Function to delete element from the end

```
void delete() {

    if (n == 0) {

        printf("\nArray is empty!\n");

        return;

    }

    printf("\nDeleted %d from the end.\n", arr[n-1]);

    n--;

}
```

// Function to display array elements

```
void display() {

    if (n == 0) {

        printf("\nArray is empty!\n");

        return;

    }

    printf("\nArray elements: ");

    for (int i = 0; i < n; i++) {

        printf("%d ", arr[i]);

    }

}
```

```
}  
  
printf("\n");  
  
}  
  
// Function to search element  
  
void search(int val) {  
  
    for (int i = 0; i < n; i++) {  
  
        if (arr[i] == val) {  
  
            printf("\nElement %d found at position %d.\n", val, i);  
  
            return;  
  
        }  
  
    }  
  
    printf("\nElement %d not found!\n", val);  
  
}
```

```
// Function to add two arrays  
  
void addArrays(int a[], int b[], int size) {  
  
    int c[MAX];  
  
    printf("\nSum of arrays: ");  
  
    for (int i = 0; i < size; i++) {  
  
        c[i] = a[i] + b[i];  
  
        printf("%d ", c[i]);  
  
    }  
  
    printf("\n");  
  
}  
  
int main() {
```

```
int choice, val, size;

while (1) {

printf("\n--- Array Operations ---\n");

printf("1. Insert at end\n");

printf("2. Delete from end\n");

printf("3. Display\n");

printf("4. Search\n");

printf("5. Add two arrays\n");

printf("6. Exit\n");

printf("Enter choice: ");

scanf("%d", &choice);

switch (choice) {

case 1:

printf("Enter value to insert: ");

scanf("%d", &val);

insert(val);

break;

case 2:

delete();

break;

case 3:

display();

break;
```

case 4:

```
printf("Enter value to search: ");
```

```
scanf("%d", &val);
```

```
search(val);
```

```
break;
```

case 5:

```
printf("Enter size of arrays: ");
```

```
scanf("%d", &size);
```

```
if (size > MAX) {
```

```
printf("Size too large!\n");
```

```
break;
```

```
}
```

```
int a[MAX], b[MAX];
```

```
printf("Enter elements of first array:\n");
```

```
for (int i = 0; i < size; i++) scanf("%d", &a[i]);
```

```
printf("Enter elements of second array:\n");
```

```
for (int i = 0; i < size; i++) scanf("%d", &b[i]);
```

```
addArrays(a, b, size);
```

```
break;
```

case 6:

```
return 0;
```

default:

```
printf("Invalid choice!\n");
```

```
}
```

```
}  
}
```

Output

--- Array Operations ---

1. Insert at end
2. Delete from end
3. Display
4. Search
5. Add two arrays
6. Exit

Enter choice: 1

Enter value to insert: 1

Inserted 1 at the end.

--- Array Operations ---

1. Insert at end
2. Delete from end
3. Display
4. Search
5. Add two arrays
6. Exit

Enter choice: 1

Enter value to insert: 5

Inserted 5 at the end.

--- Array Operations ---

1. Insert at end
2. Delete from end
3. Display
4. Search
5. Add two arrays
6. Exit

Enter choice: 1

Enter value to insert: 6

Inserted 6 at the end.

--- Array Operations ---

1. Insert at end
2. Delete from end
3. Display
4. Search
5. Add two arrays
6. Exit

Enter choice: 1

Enter value to insert: 10

Inserted 10 at the end.

--- Array Operations ---

1. Insert at end
2. Delete from end
3. Display
4. Search
5. Add two arrays
6. Exit

Enter choice: 3

Array elements: 1 5 6 10

--- Array Operations ---

1. Insert at end
2. Delete from end
3. Display
4. Search
5. Add two arrays
6. Exit

Enter choice: 2

Deleted 10 from the end.

--- Array Operations ---

1. Insert at end
2. Delete from end
3. Display
4. Search
5. Add two arrays
6. Exit

Enter choice: 4

Enter value to search: 5

Element 5 found at position 1.

--- Array Operations ---

1. Insert at end
2. Delete from end
3. Display
4. Search
5. Add two arrays
6. Exit

Enter choice: 5

Enter size of arrays: 3

Enter elements of first array:

1

3

6

Enter elements of second array:

2

9

4

Sum of arrays: 3 12 10

--- Array Operations ---

1. Insert at end

2. Delete from end

3. Display

4. Search

5. Add two arrays

6. Exit

Enter choice: 12

Invalid choice!

--- Array Operations ---

1. Insert at end

2. Delete from end

3. Display

4. Search

5. Add two arrays

6. Exit

Enter choice: 4

Enter value to search: 4

Element 4 not found!

Program 2

Aim: Add and multiply two matrices

Code

```
#include <stdio.h>

int main() {

    int m, n, p, q;

    int i, j, k, choice;

    // Input dimensions of first matrix

    printf("Enter rows and columns of first matrix: ");

    scanf("%d %d", &m, &n);

    int A[m][n];

    printf("Enter elements of first matrix:\n");

    for (i = 0; i < m; i++) {

        for (j = 0; j < n; j++) {

            scanf("%d", &A[i][j]);

        }

    }

}
```

```
// Input dimensions of second matrix

printf("Enter rows and columns of second matrix: ");

scanf("%d %d", &p, &q);

int B[p][q];

printf("Enter elements of second matrix:\n");

for (i = 0; i < p; i++) {
    for (j = 0; j < q; j++) {
        scanf("%d", &B[i][j]);
    }
}

// Display matrices

printf("\nFirst Matrix:\n");

for (i = 0; i < m; i++) {
    for (j = 0; j < n; j++) {
        printf("%d ", A[i][j]);
    }
    printf("\n");
}

printf("\nSecond Matrix:\n");

for (i = 0; i < p; i++) {
    for (j = 0; j < q; j++) {
        printf("%d ", B[i][j]);
    }
    printf("\n");
}
```

```

}

// Menu

printf("\nChoose an operation:\n");

printf("1. Add Matrices\n");

printf("2. Multiply Matrices\n");

printf("Enter choice: ");

scanf("%d", &choice);

switch (choice) {

case 1:

if (m == p && n == q) {

int Sum[m][n];

printf("\nResult of Addition:\n");

for (i = 0; i < m; i++) {

for (j = 0; j < n; j++) {

Sum[i][j] = A[i][j] + B[i][j];

printf("%d ", Sum[i][j]);

}

printf("\n");

}

} else {

printf("\nMatrix addition not possible (dimensions must match).\n");

}

break;

case 2:

```

```

if (n == p) {
int Prod[m][q];

printf("\nResult of Multiplication:\n");

for (i = 0; i < m; i++) {
for (j = 0; j < q; j++) {

Prod[i][j] = 0;

for (k = 0; k < n; k++) {

Prod[i][j] += A[i][k] * B[k][j];

}

printf("%d ", Prod[i][j]);

}

printf("\n");

}

} else {

printf("\nMatrix multiplication not possible (columns of first must equal rows of second)\n");

}

break;

default:

printf("\nInvalid choice.\n");

}

return 0;

}

```

Output

Enter rows and columns of first matrix: 2

2

Enter elements of first matrix:

1

2

3

4

Enter rows and columns of second matrix: 2

2

Enter elements of second matrix:

5

6

7

8

First Matrix:

1 2

3 4

Second Matrix:

5 6

7 8

Choose an operation:

1. Add Matrices

2. Multiply Matrices

Enter choice: 1

Result of Addition:

6 8

10 12

Choose an operation:

1. Add Matrices

2. Multiply Matrices

Enter choice: 2

Result of Multiplication:

19 22

43 50

Program 3

Aim: Implement singly linked list operations such as

a. Create a list

b. Display

c. Insertion

d. Deletion

e. Search

Code

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
// Node structure
```

```
struct Node {
```

```
    int data;
```

```
    struct Node* next;
```

```
};
```

```
// Head pointer
```

```
struct Node* head = NULL;
```

```
// Create new node
```

```
struct Node* createNode(int data) {
```

```
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
```

```
    newNode->data = data;
```

```
    newNode->next = NULL;
```

```
    return newNode;
```

```
}
```

```
// Insert at end
```

```
void insert(int data) {
```

```
    struct Node* newNode = createNode(data);
```

```
    if (head == NULL) {
```

```
        head = newNode;
```

```
    } else {
```

```
struct Node* temp = head;
while (temp->next != NULL)
temp = temp->next;
temp->next = newNode;
}
}
```

// Display list

```
void display() {
if (head == NULL) {
printf("List is empty\n");
return;
}
struct Node* temp = head;
printf("List: ");
while (temp != NULL) {
printf("%d -> ", temp->data);
temp = temp->next;
}
printf("NULL\n");
}
```

// Delete a node by value

```
void deleteNode(int key) {
struct Node* temp = head;
struct Node* prev = NULL;
```

```

if (temp != NULL && temp->data == key) {

head = temp->next;

free(temp);

return;

}

while (temp != NULL && temp->data != key) {

prev = temp;

temp = temp->next;

}

if (temp == NULL) {

printf("Value not found\n");

return;

}

prev->next = temp->next;

free(temp);

}

```

// Search element

```

void search(int key) {

struct Node* temp = head;

int pos = 1;

while (temp != NULL) {

if (temp->data == key) {

printf("Element %d found at position %d\n", key, pos);

return;

}

temp = temp->next;

pos++;

}

}

```

```
}  
  
temp = temp->next;  
  
pos++;  
  
}  
  
printf("Element %d not found\n", key);  
  
}
```

// Main

```
int main() {  
  
    int choice, value;  
  
    while (1) {  
  
        printf("\n--- Singly Linked List Menu ---\n");  
  
        printf("1. Insert\n2. Display\n3. Delete\n4. Search\n 5. Exit\n");  
  
        printf("Enter your choice: ");  
  
        scanf("%d", &choice);  
  
        switch (choice) {  
  
            case 1:  
  
                printf("Enter value to insert: ");  
  
                scanf("%d", &value);  
  
                insert(value);  
  
                break;  
  
            case 2:  
  
                display();  
  
                break;  
  
            case 3:
```

```
printf("Enter value to delete: ");  
  
scanf("%d", &value);  
  
deleteNode(value);  
  
break;  
  
case 4:  
  
printf("Enter value to search: ");  
  
scanf("%d", &value);  
  
search(value);  
  
break;  
  
case 5:  
  
exit(0);  
  
default:  
  
printf("Invalid choice\n");  
  
}  
  
}  
  
return 0;  
  
}
```

Output

--- Singly Linked List Menu ---

1. Insert
2. Display
3. Delete
4. Search

5. Exit

Enter your choice: 1

Enter value to insert: 10

--- Singly Linked List Menu ---

1. Insert

2. Display

3. Delete

4. Search

5. Exit

Enter your choice: 1

Enter value to insert: 20

--- Singly Linked List Menu ---

1. Insert

2. Display

3. Delete

4. Search

5. Exit

Enter your choice: 1

Enter value to insert: 30

--- Singly Linked List Menu ---

1. Insert

2. Display

3. Delete

4. Search

5. Exit

Enter your choice: 1

Enter value to insert: 40

--- Singly Linked List Menu ---

1. Insert

2. Display

3. Delete

4. Search

5. Exit

Enter your choice: 2

List: 10 -> 20 -> 30 -> 40 -> NULL

--- Singly Linked List Menu ---

1. Insert

2. Display

3. Delete

4. Search

5. Exit

Enter your choice: 1

Enter value to insert: 40

--- Singly Linked List Menu ---

1. Insert
2. Display
3. Delete
4. Search
5. Exit

Enter your choice: 2

List: 10 -> 20 -> 30 -> 40 -> 40 -> NULL

--- Singly Linked List Menu ---

1. Insert
2. Display
3. Delete
4. Search
5. Exit

Enter your choice: 3

Enter value to delete: 40

--- Singly Linked List Menu ---

1. Insert
2. Display
3. Delete
4. Search

5. Exit

Enter your choice: 2

List: 10 -> 20 -> 30 -> 40 -> NULL

--- Singly Linked List Menu ---

1. Insert

2. Display

3. Delete

4. Search

5. Exit

Enter your choice: 4

Enter value to search: 20

Element 20 found at position 3

--- Singly Linked List Menu ---

1. Insert

2. Display

3. Delete

4. Search

5. Exit

Enter your choice: 5

Program 4

Aim: Implement doubly linked list operations such as

- a. Create a list
- b. Display
- c. Insertion
- d. Deletion

Code

```
#include <stdio.h>

#include <stdlib.h>

// Node structure

struct Node {

    int data;

    struct Node* prev;

    struct Node* next;

};

struct Node* head = NULL;

// Function to create a new node

struct Node* createNode(int data) {

    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));

    newNode->data = data;

    newNode->prev = NULL;

    newNode->next = NULL;

    return newNode;
```

```
}
```

// a. Create a list (append nodes)

```
void createList(int data) {  
    struct Node* newNode = createNode(data);  
    if (head == NULL) {  
        head = newNode;  
        return;  
    }  
    struct Node* temp = head;  
    while (temp->next != NULL)  
        temp = temp->next;  
    temp->next = newNode;  
    newNode->prev = temp;  
}
```

// b. Display list

```
void display() {  
    struct Node* temp = head;  
    if (head == NULL) {  
        printf("List is empty\n");  
        return;  
    }  
    printf("List: ");
```

```

while (temp != NULL) {

    printf("%d <-> ", temp->data);

    temp = temp->next;

}

printf("NULL\n");

}

```

// c. Insertion at given position

```

void insertAt(int data, int pos) {

    struct Node* newNode = createNode(data);

    if (pos == 1) {

        newNode->next = head;

        if (head != NULL)

            head->prev = newNode;

        head = newNode;

        return;

    }

    struct Node* temp = head;

    for (int i = 1; temp != NULL && i < pos - 1; i++) {

        temp = temp->next;

    }

    if (temp == NULL) {

        printf("Position out of range!\n");

        return;

    }

```

```

    }
    newNode->next = temp->next;
    if (temp->next != NULL)
        temp->next->prev = newNode;
    temp->next = newNode;
    newNode->prev = temp;
}

```

// d. Deletion at given position

```

void deleteAt(int pos) {
    if (head == NULL) {
        printf("List is empty!\n");
        return;
    }
    struct Node* temp = head;
    if (pos == 1) {
        head = head->next;
        if (head != NULL)
            head->prev = NULL;
        free(temp);
        return;
    }
    for (int i = 1; temp != NULL && i < pos; i++) {
        temp = temp->next;
    }
}

```

```

    }

    if (temp == NULL) {

        printf("Position out of range!\n");

        return;

    }

    if (temp->prev != NULL)

        temp->prev->next = temp->next;

    if (temp->next != NULL)

        temp->next->prev = temp->prev;

    free(temp);

}

```

// Main function to test

```

int main() {

    int choice, data, pos, key;

    while (1) {

        printf("\n--- Doubly Linked List Menu ---\n");

        printf("1. Create (Append)\n");

        printf("2. Display\n");

        printf("3. Insert at Position\n");

        printf("4. Delete at Position\n");

        printf("5. Exit\n");

        printf("Enter choice: ");
    }
}

```

```
scanf("%d", &choice);
```

```
switch (choice) {
```

```
    case 1:
```

```
        printf("Enter data: ");
```

```
        scanf("%d", &data);
```

```
        createList(data);
```

```
        break;
```

```
    case 2:
```

```
        display();
```

```
        break;
```

```
    case 3:
```

```
        printf("Enter data & position: ");
```

```
        scanf("%d %d", &data, &pos);
```

```
        insertAt(data, pos);
```

```
        break;
```

```
    case 4:
```

```
        printf("Enter position: ");
```

```
        scanf("%d", &pos);
```

```
        deleteAt(pos);
```

```
        break;
```

```
    case 5:
```

```
        exit(0);
```

```
    default:
```

```
        printf("Invalid choice!\n");
    }
}

return 0;
}
```

Output

--- Doubly Linked List Menu ---

1. Create (Append)
2. Display
3. Insert at Position
4. Delete at Position
5. Exit

Enter choice: 1

Enter data: 10

--- Doubly Linked List Menu ---

1. Create (Append)
2. Display
3. Insert at Position
4. Delete at Position
5. Exit

Enter choice: 1

Enter data: 20

--- Doubly Linked List Menu ---

1. Create (Append)
2. Display
3. Insert at Position
4. Delete at Position
5. Exit

Enter choice: 1

Enter data: 30

--- Doubly Linked List Menu ---

1. Create (Append)
2. Display
3. Insert at Position
4. Delete at Position
5. Exit

Enter choice: 1

Enter data: 40

--- Doubly Linked List Menu ---

1. Create (Append)
2. Display
3. Insert at Position
4. Delete at Position
5. Exit

Enter choice: 1

Enter data: 50

--- Doubly Linked List Menu ---

1. Create (Append)

2. Display

3. Insert at Position

4. Delete at Position

5. Exit

Enter choice: 1

Enter data: 60

--- Doubly Linked List Menu ---

1. Create (Append)

2. Display

3. Insert at Position

4. Delete at Position

5. Exit

Enter choice: 2

List: 10 <-> 20 <-> 30 <-> 40 <-> 50 <-> 60 <-> NULL

--- Doubly Linked List Menu ---

1. Create (Append)

2. Display

3. Insert at Position

4. Delete at Position

5. Exit

Enter choice: 3

Enter data & position: 25

2

--- Doubly Linked List Menu ---

1. Create (Append)

2. Display

3. Insert at Position

4. Delete at Position

5. Exit

Enter choice: 2

List: 10 <-> 25 <-> 20 <-> 30 <-> 40 <-> 50 <-> 60 <-> NULL

--- Doubly Linked List Menu ---

1. Create (Append)

2. Display

3. Insert at Position

4. Delete at Position

5. Exit

Enter choice: 4

Enter position: 2

--- Doubly Linked List Menu ---

1. Create (Append)
2. Display
3. Insert at Position
4. Delete at Position
5. Exit

Enter choice: 2

List: 10 <-> 20 <-> 30 <-> 40 <-> 50 <-> 60 <-> NULL

--- Doubly Linked List Menu ---

1. Create (Append)
2. Display
3. Insert at Position
4. Delete at Position
5. Exit

Enter choice: 5

Program 5

Aim: Implement a stack using arrays.

Code

Implementation of Stack using Array in C

```
#include <stdio.h>
```

```
#define SIZE 8
```

```
int stack[SIZE];
```

```
int top = -1;
```

```
void push(int value) {
```

```
    if (top >= SIZE - 1) {
```

```
        printf("Stack is full! Cannot push %d\n", value);
```

```
        return;
```

```
    }
```

```
    stack[++top] = value;
```

```
    printf("%d pushed to stack.\n", value);
```

```
}
```

```
int pop() {
```

```
    if (top < 0) {
```

```
        printf("Stack is empty! Cannot pop.\n");
```

```
        return -1;
```

```
    }
```

```
    return stack[top--];
```

```
}
```

```
int peek() {
```

```
    if (top < 0) {
```

```
        printf("Stack is empty!\n");
```

```
        return -1;
    }
    return stack[top];
}
```

```
void display() {
    if (top < 0) {
        printf("Stack is empty.\n");
        return;
    }
    printf("Stack elements: ");
    for (int i = top; i >= 0; i--)
        printf("%d ", stack[i]);
    printf("\n");
}
```

```
int main() {
    int choice, value;
    while (1) {
        printf("\nStack Menu:\n");
        printf("1. Push\n");
        printf("2. Pop\n");
        printf("3. Peek\n");
        printf("4. Display\n");
```

```
printf("5. Exit\n");

printf("Enter your choice: ");

scanf("%d", &choice);


switch (choice) {

    case 1:

        printf("Enter value to push: ");

        scanf("%d", &value);

        push(value);

        break;

    case 2:

        value = pop();

        if (value != -1)

            printf("Popped: %d\n", value);

        break;

    case 3:

        value = peek();

        if (value != -1)

            printf("Top element: %d\n", value);

        break;

    case 4:

        display();

        break;

    case 5:
```

```
printf("Exiting...\n");
```

```
return 0;
```

default:

```
printf("Invalid choice. Try again.\n");
```

```
}
```

```
}
```

```
}
```

Output

Stack Menu:

1. Push

2. Pop

3. Peek

4. Display

5. Exit

Enter your choice: 1

Enter value to push: 10

10 pushed to stack.

Stack Menu:

1. Push

2. Pop

3. Peek

4. Display

5. Exit

Enter your choice: 1

Enter value to push: 20

20 pushed to stack.

Stack Menu:

1. Push

2. Pop

3. Peek

4. Display

5. Exit

Enter your choice: 1

Enter value to push: 30

30 pushed to stack.

Stack Menu:

1. Push

2. Pop

3. Peek

4. Display

5. Exit

Enter your choice: 1

Enter value to push: 40

40 pushed to stack.

Stack Menu:

1. Push
2. Pop
3. Peek
4. Display
5. Exit

Enter your choice: 1

Enter value to push: 50

50 pushed to stack.

Stack Menu:

1. Push
2. Pop
3. Peek
4. Display
5. Exit

Enter your choice: 1

Enter value to push: 60

60 pushed to stack.

Stack Menu:

1. Push
2. Pop
3. Peek

4. Display

5. Exit

Enter your choice: 2

Popped: 60

Stack Menu:

1. Push

2. Pop

3. Peek

4. Display

5. Exit

Enter your choice: 3

Top element: 50

Stack Menu:

1. Push

2. Pop

3. Peek

4. Display

5. Exit

Enter your choice: 4

Stack elements: 50 40 30 20 10

Stack Menu:

1. Push

2. Pop

3. Peek

4. Display

5. Exit

Enter your choice: 5

Exiting...

Program 6

Aim: Implement a queue using arrays.

Code

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#include <stdlib.h>
```

```
#include <stdbool.h>
```

```
#define MAX 6
```

```
int intArray[MAX];
```

```
int front = 0;
```

```
int rear = -1;
```

```
int itemCount = 0;
```

```
int peek()
```

```
{
```

```
        return intArray[front];
    }

    bool isEmpty()
    {
        return itemCount == 0;
    }

    bool isFull()
    {
        return itemCount == MAX;
    }

    int size()
    {
        return itemCount;
    }

    void insert(int data)
    {
        if(!isFull())
        {
            if(rear == MAX-1)
            {
                rear = -1;
            }
            intArray[++rear] = data;
            itemCount++;
        }
    }
}
```

```

}

}

int removeData()

{
    int data = intArray[front++];

    if(front == MAX)

    front = 0;

    itemCount--;

    return data;

}

```

```

int main() {

```

```

/* insert 5 items */

```

```

insert(3);

```

```

insert(5);

```

```

insert(9);

```

```

insert(1);

```

```

insert(12);

```

```

// front : 0

```

```

// rear : 4

```

```

// -----

```

```

// index : 0 1 2 3 4

```

```

// -----

```

```

// queue : 3 5 9 1 12

```

```

insert(15);

// front : 0

// rear : 5

// -----

// index : 0 1 2 3 4 5

// -----

// queue : 3 5 9 1 12 15

if(isFull()){

    printf("Queue is full!\n");

}

// remove one item

int num = removeData();

printf("Element removed: %d\n",num);

// front : 1

// rear : 5

// -----

// index : 1 2 3 4 5

// -----

// queue : 5 9 1 12 15

// insert more items

insert(16);

// front : 1

// rear : -1

// -----

```

```

// index : 0 1 2 3 4 5

// -----

// queue : 16 5 9 1 12 15

// As queue is full, elements will not be inserted.

insert(17);

insert(18);

// -----

// index : 0 1 2 3 4 5

// -----

// queue : 16 5 9 1 12 15

printf("Element at front: %d\n",peek());

printf("-----\n");

printf("index : 5 4 3 2 1 0\n");

printf("-----\n");

printf("Queue: ");

while(!isEmpty()) {

    int n = removeData();

    printf("%d ",n);

}

}

```

Output

Queue is full!

Element removed: 3

Element at front: 5

index : 5 4 3 2 1 0

Queue: 5 9 1 12 15 16

Program 7

Aim: To create a Binary Search Tree using array

Program 8

Aim: Check a given binary tree is BST or not

Program 9

Aim: Implement an undirected graph and

- a. Count the number of edges
- b. Search

Program 10

Aim: Implement a directed graph and

- a. Count the number of edges
- b. Search

